InterviewIqAI

Phase 0
what is software Engineering

Software Engineering is the systematic process of planning, designing, building, testing, deploying, and maintaining software to solve real-world problems.

Applying software engineering to interview IqAI

 How will users sign in?

 Where will resumes be stored?

 How will AI evaluate answers?

 Which database should hold interview history?

 How will the frontend talk to the backend?

Architecture

🌐 User │ ▼ Next.js Frontend │ HTTP / REST APIs │ ▼ FastAPI Backend ┌──────────────┼ ──────────────┐ │ │ │ ▼ ▼ ▼ PostgreSQL Gemini AI ChromaDB

"InterviewIQ follows a client-server architecture. The frontend is built with Next.js and communicates with a FastAPI backend through REST APIs over HTTP. The backend handles authentication, business logic, and AI orchestration. Structured data is stored in PostgreSQL, semantic embeddings are stored in ChromaDB, and Gemini is used for AI-powered interview generation and evaluation."

Requirement Gatherings:

InterviewIQ is an AI-powered interview preparation platform that helps students improve placement readiness through personalized, resume-aware mock interviews, coding assessments, skill-gap analysis, and AI-driven learning recommendations. The platform analyzes resumes, generates role-specific interview questions, evaluates responses, identifies missing skills for target job roles, recommends learning paths, and tracks progress through detailed analytics dashboards. The system aims to provide a comprehensive interview preparation ecosystem that bridges the gap between academic learning and industry expectations.

My proposed final feature set:

Authentication → Resume Upload → AI Interviews → AI Evaluation → Dashboard → Coding Interview Module → Skill Gap Analysis → Personalized Learning Suggestions

User stories

Authentication

Story 1

As a student, I want to create an account so that I can access InterviewIQ.

Story 2

As a student, I want to log in securely so that I can access my interviews and reports.

Resume Management

Story 3

As a student, I want to upload my resume so that the AI can generate personalized interview questions.

Story 4

As a student, I want to update my resume so that future interviews reflect my latest skills.

Interview Generation

Story 5

As a student, I want to choose a target role so that the interview matches my career goals.

Example:

Backend Developer

Frontend Developer

Full Stack Developer

Data Analyst

AI Engineer

Story 6

As a student, I want AI-generated questions based on my resume so that the interview feels realistic.

Story 7

As a student, I want to select interview difficulty so that I can practice at my level.

Example:

Easy

Medium

Hard

Evaluation

Story 8

As a student, I want my answers evaluated automatically so that I can identify weaknesses.

Story 9

As a student, I want detailed feedback so that I know how to improve.

Dashboard

Story 10

As a student, I want to view previous interviews so that I can track progress.

Story 11

As a student, I want to see my scores and analytics so that I can measure improvement over time.

Skill Gap Analysis

Story 12

As a student, I want the system to compare my skills with job requirements so that I know what skills are missing.

Learning Recommendations

Story 13

As a student, I want learning recommendations so that I can improve weak areas.

Coding Assessment

Story 14

As a student, I want coding interview challenges so that I can prepare for technical rounds.

Story 15

As a student, I want coding solutions evaluated so that I can understand mistakes.

Admin User Stories

Admin is simpler.

Story 16

As an admin, I want to view platform statistics so that I can monitor usage.

Story 17

As an admin, I want to manage users so that inappropriate accounts can be handled.

What is a Use Case?

A User Story tells us:

What the user wants.

A Use Case tells us:

How it happens step by step

Example Use Case

Use Case: Generate AI Interview

Actor

Student

Preconditions

User Registered

User Logged In

Resume Uploaded

Main Flow

1. Student opens dashboard

2. Student selects role

3. Student selects difficulty

4. Student clicks Generate Interview

5. System reads resume

6. System creates AI prompt

7. Gemini generates questions

8. Questions are saved

9. Questions displayed

Success Outcome

Interview Created Successfully

Use Case: Evaluate Answers

Actor

Student

Preconditions

Interview Exists

Main Flow

1. Student submits answers

2. System sends answers to AI

3. AI evaluates answers

4. System generates feedback

5. Results stored

6. Dashboard updated

Success Outcome

Feedback Generated

Why User Stories Matter

Look at what we've already discovered.

From the stories alone, we know we need:

Users

Resumes

Interviews

Questions

Answers

Reports

Analytics

Coding Challenges

Skill Gap Analysis

Learning Recommendations

See what's happening?

We're naturally discovering our database entities.

This is exactly how architects derive database design.

Feature Hierarchy

Let's organize InterviewIQ features.

InterviewIQ

```
│
├── Authentication
│   ├── Signup
│   ├── Login
│   └── Logout
│
├── Resume
│   ├── Upload
│   ├── Parse
│   └── Update
│
├── Interview
│   ├── Generate
│   ├── Answer
│   └── Evaluate
│
├── Dashboard
│   ├── History
│   ├── Reports
│   └── Analytics
│
```

├── Skill Gap Analysis

│

├── Learning Recommendations

│

└── Coding Assessment

This hierarchy will soon become:

- Database tables
- Backend modules
- API routes
- Frontend pages

🎯 **Deliverable from Module 2**

We have now identified:

Primary Actor

Student

Secondary Actor

Admin

Core Features

Authentication

Resume Management

AI Interviews

Answer Evaluation

Dashboard

Skill Gap Analysis

Learning Recommendations

Coding Assessment

Core Use Cases

Signup

Login

Upload Resume

Generate Interview

Submit Answers

Get Feedback

View Dashboard

Analyze Skills

Get Learning Suggestions

Attempt Coding Challenge

Module 3:

Module 3: Feature Breakdown & Priority Matrix

Objective

The purpose of this module is to define the scope of InterviewIQ by prioritizing features based on their importance and feasibility. The MoSCoW prioritization technique is used to categorize features into Must Have, Should Have, Could Have, and Won't Have categories.

1. Must Have Features (Core MVP)

These features are essential for the system to achieve its primary objective of providing AI-powered interview preparation.

Authentication Module

- User Registration
- User Login
- User Logout
- JWT-based Authentication and Authorization

Resume Management Module

- Resume Upload (PDF)
- Resume Parsing and Text Extraction
- Resume Storage and Management

AI Interview Generation Module

- Role Selection
- Difficulty Selection
- AI-based Question Generation

Answer Evaluation Module

- Answer Submission
- AI-based Evaluation
- Feedback Generation

Dashboard & Analytics Module

- Interview History
- Performance Scores
- Feedback Reports
- Progress Tracking

2. Should Have Features

These features significantly enhance the platform and improve the learning experience.

Skill Gap Analysis Module

- Analyze user skills from resume
- Compare skills against target job role requirements
- Identify missing skills and competencies

Personalized Learning Recommendation Module

- Analyze weak areas based on interview performance
- Recommend learning topics and improvement areas
- Suggest relevant resources for skill development

Coding Assessment Module

- Coding Interview Questions
- Code Submission Interface
- Test Case Evaluation
- AI-generated Coding Feedback

3. Could Have Features

These features provide additional value but are not critical for the initial release.

Resume Scoring

- AI-generated resume quality score
- Resume improvement suggestions

Difficulty Recommendation System

- Automatically recommend interview difficulty based on previous performance

Gamification Features

- Leaderboards
- Achievement Badges
- Progress Milestones

4. Won't Have Features (Version 1)

The following features are excluded from the initial version due to increased development complexity and time constraints.

Voice-Based Interview System

- Speech-to-Text Processing
- Text-to-Speech Responses
- Real-Time Voice Interaction

Video Interview Analysis

- Webcam Integration
- Facial Expression Analysis
- Emotion Recognition

Recruiter Portal

- Candidate Management
- Recruiter Dashboard
- Hiring Analytics

Job Marketplace

- Job Listings
- Candidate Matching
- Application Tracking

5. Final Project Scope

The finalized scope for InterviewIQ Version 1 includes:

1. Authentication System
2. Resume Upload and Parsing
3. AI Interview Generation
4. AI Answer Evaluation
5. Dashboard and Analytics
6. Skill Gap Analysis
7. Personalized Learning Recommendations
8. Coding Assessment Module

6. System Modules

Based on the finalized scope, the system is divided into the following major modules:

Module 1: Authentication

Responsibilities:

- User Registration
- User Login
- JWT Authentication
- Route Protection

Module 2: Resume Processing

Responsibilities:

- Resume Upload
- Resume Parsing
- Resume Storage
- Skill Extraction

Module 3: Interview Engine

Responsibilities:

- Interview Generation
- Question Management
- Interview Session Handling
- Response Collection

Module 4: Evaluation Engine

Responsibilities:

- Answer Evaluation
- Feedback Generation
- Performance Scoring

Module 5: Analytics Engine

Responsibilities:

- Progress Tracking
- Report Generation
- Performance Analytics

Module 6: Skill Gap Analysis Engine

Responsibilities:

- Skill Comparison

- Missing Skill Identification
- Gap Analysis Reports

Module 7: Learning Recommendation Engine

Responsibilities:

- Weakness Analysis
- Learning Recommendations
- Resource Suggestions

Module 8: Coding Assessment Engine

Responsibilities:

- Coding Challenges
- Code Execution
- Test Case Evaluation
- AI Feedback Generation

Outcome of Module 3

The feature scope and system modules of InterviewIQ have been finalized. These modules will serve as the foundation for subsequent phases, including User Flow Design, Database Design, API Design, System Architecture, and Implementation Planning.

Module 4: User Flows & System Actors

Objective

The objective of this module is to identify the system actors and define the user interaction flows within InterviewIQ. User flow analysis helps determine the required pages, APIs, database interactions, and system behavior before implementation begins.

1. System Actors

Primary Actor: Student

The student is the main user of the platform and interacts with the core functionalities of InterviewIQ.

Responsibilities

- Register and log in to the platform
- Upload and manage resumes
- Generate AI-powered mock interviews
- Submit interview responses
- View feedback and performance reports
- Analyze skill gaps

- Access personalized learning recommendations
- Attempt coding assessments

Secondary Actor: Admin

The admin is responsible for managing and monitoring the platform.

Responsibilities

- Manage user accounts
- Monitor platform activity
- View system analytics and statistics
- Handle administrative operations

2. Student User Flow

The primary user journey within InterviewIQ follows the sequence below:

Landing Page

↓

Signup / Login

↓

Dashboard

↓

Upload Resume

↓

Resume Analysis

↓

Generate Interview

↓

Answer Questions

↓

AI Evaluation

↓

Feedback Report

↓

Analytics Dashboard

↓

Skill Gap Analysis

↓

Learning Recommendations

3. Authentication Flow

Signup Process

Student

↓

Signup Page

↓

Enter Details

↓

Validate Input

↓

Create Account

↓

Dashboard

Login Process

Student

↓

Login Page

↓

Enter Credentials

↓

Credential Validation

↓

JWT Generation

↓

Dashboard

4. Resume Management Flow

Dashboard

↓

Upload Resume

↓

PDF Validation

↓

Resume Parsing

↓

Skill Extraction

↓

Resume Storage

↓

Analysis Completed

---

5. AI Interview Flow

Select Role

↓

Select Difficulty

↓

Generate Interview

↓

Resume Analysis

↓

Prompt Generation

↓

AI Question Generation

↓

Store Questions

↓

Start Interview

6. Answer Evaluation Flow

Submit Answers

↓

AI Evaluation

↓

Performance Scoring

↓

Feedback Generation

↓

Store Results

↓

Display Report

7. Dashboard & Analytics Flow

Dashboard Request

↓

Retrieve Interview History

↓

Retrieve Scores

↓

Generate Analytics

↓

Display Dashboard

8. Skill Gap Analysis Flow

Resume Skills

+

Target Role

↓

Skill Comparison

↓

Identify Missing Skills

↓

Generate Gap Report

↓

Display Results

9. Learning Recommendation Flow

Interview Performance

+

Skill Gap Analysis

↓

Weakness Detection

↓

Recommendation Generation

↓

Learning Resources

↓

Display Suggestions

10. Coding Assessment Flow

Start Coding Assessment

↓

Generate Coding Question

↓

Code Submission

↓

Execute Test Cases

↓

Evaluate Solution

↓

Generate Feedback

↓

Store Results

11. Admin Flow

Admin Login

↓

Admin Dashboard

↓

User Management

↓

System Monitoring

↓

Analytics Review

Identified Student Pages

- Landing Page
- Signup Page
- Login Page
- Dashboard
- Resume Upload Page
- Interview Page
- Feedback Page
- Analytics Page
- Skill Gap Analysis Page
- Learning Recommendations Page
- Coding Assessment Page

Identified Admin Pages

- Admin Login Page
- Admin Dashboard
- User Management Page
- System Analytics Page

Outcome of Module 4

The system actors, user journeys, and interaction flows for InterviewIQ have been defined. These flows provide a clear understanding of how users interact with the platform and serve as the foundation for database design, API design, frontend page structure, and overall system architecture.

Now your documentation contains:

- ✅ Module 1: Requirement Gathering
- ✅ Module 2: User Stories & Use Cases
- ✅ Module 3: Feature Breakdown & Priority Matrix
- ✅ Module 4: User Flows & System Actors

The next major documentation section will be **Module 5: Database Design (ER Diagram, Entities, Relationships, Tables, Primary Keys, Foreign Keys)**, which is usually one of the most important sections in a final-year project report.

Module 5: Database Design

Objective

The objective of this module is to design the database structure for InterviewIQ by identifying entities, defining relationships, and creating a normalized schema that supports all system functionalities. PostgreSQL is selected as the primary relational database due to its strong support for structured data, relationships, scalability, and reliability.

---

1. Database Entities

Based on the system requirements and user flows, the following core entities have been identified:

- User
- Password Reset
- Resume
- Interview
- Question
- Answer
- Report
- Skill Gap Analysis
- Learning Recommendation
- Coding Assessment

Each entity represents a major component of the system and will be implemented as a separate database table.

---

2. Entity Relationships

The relationships between the entities are defined as follows:

User → Resume

- One User can have multiple Resumes.
- Relationship: One-to-Many (1:N)

Resume → Interview

- One Resume can be used for multiple Interviews.
- One Interview is generated from a specific Resume.
- Relationship: One-to-Many (1:N)

User → Interview

- One User can participate in multiple Interviews.
- Relationship: One-to-Many (1:N)

Interview → Question

- One Interview contains multiple Questions.
- Relationship: One-to-Many (1:N)

Question → Answer

- One Question has one corresponding Answer.
- Relationship: One-to-One (1:1)

Interview → Report

- One Interview generates one Performance Report.
- Relationship: One-to-One (1:1)

User → Skill Gap Analysis

- One User can generate multiple Skill Gap Analysis records.
- Relationship: One-to-Many (1:N)

User → Learning Recommendation

- One User can receive multiple Learning Recommendations.
- Relationship: One-to-Many (1:N)

User → Coding Assessment

- One User can attempt multiple Coding Assessments.
- Relationship: One-to-Many (1:N)

User → Password Reset

- One User can generate multiple password reset requests.
- Relationship: One-to-Many (1:N)

---

3. Database Schema

Users Table

Stores account and authentication information.

Field Name	Data Type
id	UUID (Primary Key)
name	VARCHAR
email	VARCHAR (Unique)
password_hash	VARCHAR
role	VARCHAR
created_at	TIMESTAMP

Password_Resets Table

Stores password reset requests securely.

Field Name	Data Type
id	UUID (Primary Key)
user_id	UUID (Foreign Key)
token_hash	VARCHAR
expires_at	TIMESTAMP
used	BOOLEAN
created_at	TIMESTAMP

Purpose:

Supports secure password recovery functionality. Only token hashes are stored rather than raw tokens to improve security.

Resumes Table

Stores uploaded resumes and extracted content.

Field Name	Data Type
id	UUID (Primary Key)
user_id	UUID (Foreign Key)
file_url	TEXT
extracted_text	TEXT

Field Name	Data Type
uploaded_at	TIMESTAMP

Interviews Table

Stores interview session information.

Field Name	Data Type
id	UUID (Primary Key)
user_id	UUID (Foreign Key)
resume_id	UUID (Foreign Key)
role	VARCHAR
difficulty	VARCHAR
created_at	TIMESTAMP

Purpose:

The resume_id field links every interview to the specific resume used during interview generation, ensuring traceability and resume-aware interview creation.

Questions Table

Stores AI-generated interview questions.

Field Name	Data Type
id	UUID (Primary Key)
interview_id	UUID (Foreign Key)
question_text	TEXT
category	VARCHAR

Answers Table

Stores user responses and evaluation results.

Field Name	Data Type
id	UUID (Primary Key)

Field Name	Data Type
question_id	UUID (Foreign Key)
answer_text	TEXT
score	FLOAT
feedback	TEXT

Reports Table

Stores interview performance reports.

Field Name	Data Type
id	UUID (Primary Key)
interview_id	UUID (Foreign Key)
overall_score	FLOAT
strengths	TEXT
weaknesses	TEXT
created_at	TIMESTAMP

Skill_Gaps Table

Stores identified skill gaps for target roles.

Field Name	Data Type
id	UUID (Primary Key)
user_id	UUID (Foreign Key)
target_role	VARCHAR
missing_skills	JSONB

Learning_Recommendations Table

Stores personalized learning suggestions.

Field Name	Data Type
id	UUID (Primary Key)

Field Name	Data Type
user_id	UUID (Foreign Key)
recommendation_text	TEXT
generated_at	TIMESTAMP

Coding_Assessments Table

Stores coding assessment attempts and results.

Field Name	Data Type
id	UUID (Primary Key)
user_id	UUID (Foreign Key)
problem_title	VARCHAR
submitted_code	TEXT
status	VARCHAR
execution_time_ms	INTEGER
score	FLOAT
feedback	TEXT
created_at	TIMESTAMP

Purpose:

Tracks coding submissions, execution status, performance metrics, and evaluation results.

---

4. Primary Keys and Foreign Keys

Primary Keys

Each table contains a UUID-based primary key (id) to uniquely identify records.

Advantages of UUID

- Globally unique identifiers
- Improved security by preventing predictable IDs
- Better scalability for distributed systems

Foreign Keys

Foreign keys are used to establish relationships between tables.

Examples:

- user_id in Resumes references Users(id)
- user_id in Interviews references Users(id)
- resume_id in Interviews references Resumes(id)
- interview_id in Questions references Interviews(id)
- question_id in Answers references Questions(id)
- interview_id in Reports references Interviews(id)
- user_id in Password_Resets references Users(id)

---

5. Database Normalization

The database follows normalization principles to reduce redundancy and maintain data consistency.

Instead of storing all information in a single table, data is separated into specialized tables such as:

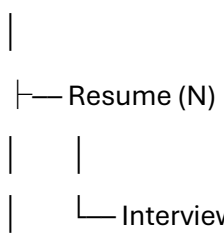
- Users
- Password Resets
- Resumes
- Interviews
- Questions
- Answers
- Reports
- Skill Gaps
- Learning Recommendations
- Coding Assessments

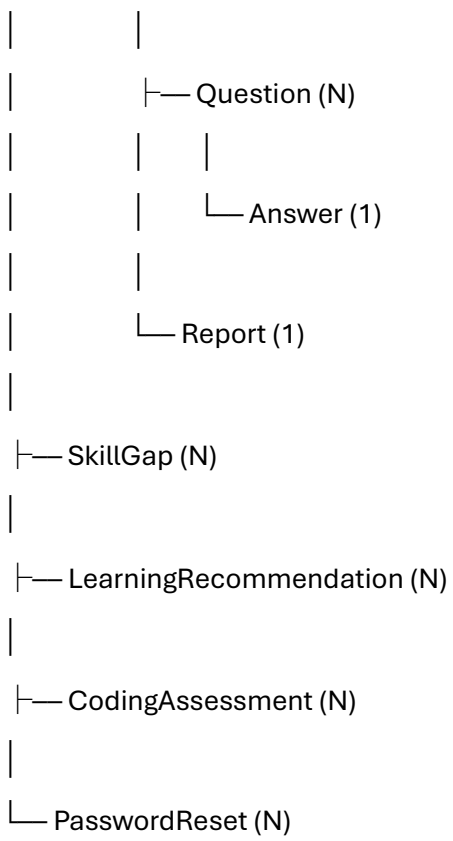
This structure improves maintainability, scalability, and query efficiency.

---

6. Entity Relationship Summary

User (1)





Outcome of Module 5

A complete relational database design has been created for InterviewIQ, including entities, relationships, table structures, primary keys, foreign keys, authentication support tables, and coding assessment tracking fields. The schema supports resume-aware interview generation, AI evaluation, coding assessments, skill-gap analysis, learning recommendations, and secure authentication workflows while ensuring scalability, maintainability, and data integrity.

Module 6: API Design

Objective

The objective of this module is to define the REST API endpoints that enable communication between the frontend, backend, database, and AI services. The API layer acts as the interface through which users interact with the InterviewIQ platform.

The system follows RESTful API design principles and uses HTTP methods such as GET, POST, PUT, and DELETE for resource management.

---

1. Authentication APIs

These APIs manage user registration, login, authentication, and password recovery.

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate user and generate JWT token
GET	/api/auth/profile	Retrieve logged-in user profile
PUT	/api/auth/profile	Update user profile
POST	/api/auth/logout	Logout user
POST	/api/auth/forgot-password	Generate password reset request
POST	/api/auth/reset-password	Reset password using valid reset token

---

2. Resume Management APIs

These APIs handle resume upload, retrieval, and management.

Method	Endpoint	Description
POST	/api/resumes/upload	Upload resume file
GET	/api/resumes	Retrieve all resumes of a user
GET	/api/resumes/{resumeId}	Retrieve a specific resume
PUT	/api/resumes/{resumeId}	Update resume
DELETE	/api/resumes/{resumeId}	Delete resume

---

3. Interview Management APIs

These APIs manage interview generation and interview sessions.

Method	Endpoint	Description
POST	/api/interviews/generate	Generate AI-based interview questions using selected resume
GET	/api/interviews	Retrieve all interview sessions
GET	/api/interviews/{interviewId}	Retrieve specific interview details
POST	/api/interviews/{interviewId}/submit	Submit interview answers
DELETE	/api/interviews/{interviewId}	Delete interview record

The interview generation request includes:

- Resume ID
- Target Role
- Difficulty Level

This ensures every interview is linked to a specific resume.

---

4. Question APIs

These APIs handle interview questions associated with interview sessions.

Method	Endpoint	Description
GET	/api/questions/{interviewId}	Retrieve interview questions
GET	/api/questions/{questionId}	Retrieve specific question

---

5. Answer Evaluation APIs

These APIs evaluate answers and generate feedback reports.

Method	Endpoint	Description
POST	/api/evaluation/evaluate	Evaluate submitted answers
GET	/api/evaluation/report/{interviewId}	Retrieve interview evaluation report

---

6. Dashboard & Analytics APIs

These APIs provide performance analytics and interview history.

Method	Endpoint	Description
GET	/api/dashboard	Retrieve dashboard overview
GET	/api/dashboard/history	Retrieve interview history
GET	/api/dashboard/analytics	Retrieve performance analytics
GET	/api/dashboard/reports	Retrieve generated reports

---

7. Skill Gap Analysis APIs

These APIs identify missing skills for a target role.

Method	Endpoint	Description
POST	/api/skill-gap/analyze	Generate skill gap analysis

Method	Endpoint	Description
GET	/api/skill-gap	Retrieve previous skill gap reports

---

8. Learning Recommendation APIs

These APIs generate and retrieve personalized learning recommendations.

Method	Endpoint	Description
POST	/api/recommendations/generate	Generate learning recommendations
GET	/api/recommendations	Retrieve recommendation history

---

9. Coding Assessment APIs

These APIs manage coding interview assessments and code execution workflows.

Method	Endpoint	Description
GET	/api/coding/questions	Retrieve coding questions
POST	/api/coding/submit	Submit coding solution
GET	/api/coding/results/{assessmentId}	Retrieve coding assessment result
GET	/api/coding/history	Retrieve coding assessment history
GET	/api/coding/status/{assessmentId}	Retrieve execution status
GET	/api/coding/output/{assessmentId}	Retrieve execution output and feedback

Coding submissions follow an asynchronous workflow:

1. Submission received
2. Job added to execution queue
3. Worker executes code inside sandbox
4. Results stored
5. Status updated
6. Frontend retrieves results

---

10. Admin APIs

These APIs are accessible only to administrators.

Method	Endpoint	Description
GET	/api/admin/users	Retrieve all users
GET	/api/admin/analytics	View platform statistics
DELETE	/api/admin/users/{userId}	Remove user account
GET	/api/admin/reports	View system reports

API Security

The system uses JWT (JSON Web Token) authentication to secure protected endpoints.

Protected API groups include:

- Resume Management
- Interview Management
- Evaluation
- Dashboard
- Skill Gap Analysis
- Recommendations
- Coding Assessments
- Admin Operations

Authorization is enforced using Role-Based Access Control (RBAC).

Additional security measures include:

- Password reset token validation
- Input validation
- Rate limiting
- Secure HTTPS communication
- Protected coding execution environment

API Response Format

Successful Response

```
{
  "success": true,
  "message": "Operation completed successfully",
```

"data": {}

}

Error Response

{

"success": false,

"message": "Error description"

}

Outcome of Module 6

A complete REST API specification has been defined for InterviewIQ. These endpoints establish communication between the frontend, backend services, AI modules, coding execution services, and database systems. The API design supports authentication, resume-aware interview generation, AI evaluation, coding assessments, skill-gap analysis, learning recommendations, and administrative operations while maintaining security, scalability, and maintainability.

.

Module 7: High-Level System Architecture

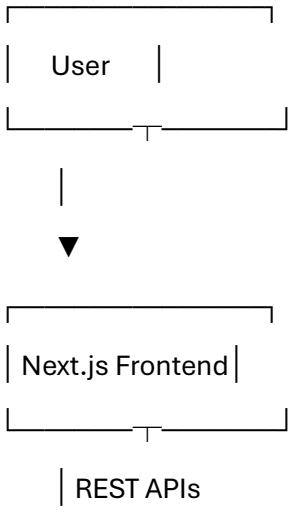
Objective

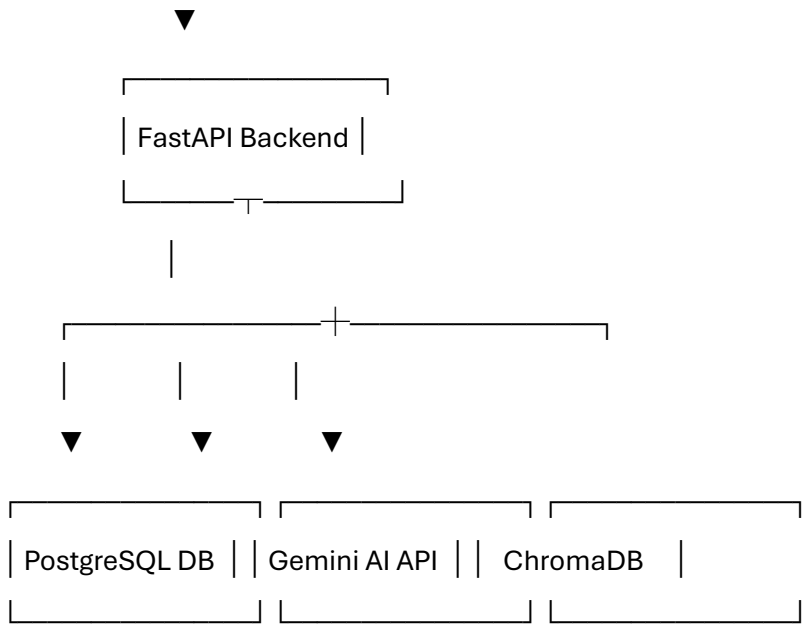
The objective of this module is to define the overall architecture of InterviewIQ and illustrate how different system components interact to provide AI-powered interview preparation, evaluation, skill-gap analysis, and learning recommendations.

The architecture follows a client-server model with separate frontend, backend, database, AI, and vector storage layers to ensure scalability, maintainability, and modularity.

---

1. High-Level Architecture Diagram





---

2. Architecture Components

A. User Layer

The user interacts with the InterviewIQ platform through a web browser.

Responsibilities:

- Registration and login
- Resume upload
- Interview participation
- Viewing reports and analytics
- Accessing coding assessments
- Receiving recommendations

B. Frontend Layer (Next.js)

The frontend is responsible for presenting information and collecting user input.

Responsibilities

- User Interface (UI)
- Authentication screens
- Dashboard and analytics visualization
- Resume upload interface
- Interview screens

- Coding assessment interface
- API communication with backend

Why Next.js?

- Server-side rendering support
- Excellent React ecosystem
- Fast page loading
- SEO optimization
- Modern routing system
- Scalable architecture

C. Backend Layer (FastAPI)

FastAPI acts as the central processing unit of the system.

Responsibilities

- Authentication and authorization
- Business logic implementation
- Resume processing
- AI prompt generation
- Evaluation workflows
- Skill gap analysis
- Recommendation generation
- API management

Why FastAPI?

- High performance
- Built-in API documentation
- Easy AI/ML integration
- Python ecosystem compatibility
- Asynchronous request handling

D. PostgreSQL Database

PostgreSQL serves as the primary relational database.

Stored Data

- Users
- Resumes
- Interviews
- Questions
- Answers
- Reports
- Skill Gap Records
- Recommendations
- Coding Assessments

Why PostgreSQL?

- Strong relational support
- ACID compliance
- Scalability
- Security
- Advanced querying capabilities

E. Gemini AI Service

Gemini provides the intelligence layer of the platform.

Responsibilities

- Interview question generation
- Answer evaluation
- Feedback generation
- Skill analysis
- Recommendation generation

Why Gemini?

- Advanced reasoning capabilities
- Natural language understanding
- High-quality content generation
- Strong support for educational applications

F. ChromaDB (Vector Database)

ChromaDB stores vector embeddings generated from resumes and learning resources.

Responsibilities

- Semantic search
- Resume understanding
- Context retrieval
- Similarity matching
- Retrieval-Augmented Generation (RAG)

Why ChromaDB?

- Lightweight vector database
- Easy integration with AI workflows
- Efficient embedding retrieval
- Open-source solution

---

3. System Workflow

User Authentication

User

↓

Frontend

↓

FastAPI

↓

PostgreSQL

↓

JWT Generated

↓

Frontend

Interview Generation Workflow

User Request

↓

Frontend

↓

FastAPI

↓

Resume Data Retrieved

↓

Gemini AI

↓

Questions Generated

↓

Stored in PostgreSQL

↓

Returned to User

Answer Evaluation Workflow

User Answers

↓

Frontend

↓

FastAPI

↓

Gemini Evaluation

↓

Score & Feedback

↓

PostgreSQL Storage

↓

Dashboard Display

Skill Gap Analysis Workflow

Resume Skills

+

Target Role

↓

FastAPI Processing

↓

Gemini Analysis

↓

Gap Identification

↓

Store Results

↓

Display Report

---

4. Security Architecture

The system incorporates multiple security mechanisms:

Authentication

- JWT-based authentication

Authorization

- Role-Based Access Control (RBAC)

Password Security

- Password hashing using bcrypt

API Security

- Protected routes
- Token validation

Data Protection

- Secure database access
- Encrypted communication using HTTPS

---

5. Scalability Considerations

The architecture is designed for future expansion.

Possible future enhancements include:

- Microservices architecture

- Dedicated recommendation service
- Voice interview support
- Video interview analysis
- Recruiter portal
- Cloud deployment and auto-scaling

---

6. Technology Stack Summary

Layer	Technology
Frontend	Next.js
Backend	FastAPI
Database	PostgreSQL
AI Engine	Gemini API
Vector Database	ChromaDB
Authentication	JWT
Password Security	bcrypt
Deployment	Cloud Platform (Future Phase)

Outcome of Module 7

A complete high-level architecture has been designed for InterviewIQ. The architecture clearly defines system components, responsibilities, communication flows, security mechanisms, and scalability considerations. This architecture serves as the blueprint for implementation and deployment of the platform.

Module 8: Low-Level Design (LLD)

Objective

The objective of this module is to define the internal structure of the InterviewIQ system. Low-Level Design focuses on how the application will be organized into modules, services, repositories, and components to ensure maintainability, scalability, and clean code practices.

The system follows a layered architecture consisting of Presentation, Business Logic, Data Access, and Database layers.

---

1. Layered Architecture

The backend follows a layered architecture to separate responsibilities.

Client Request

↓

API Routes Layer

↓

Service Layer

↓

Repository Layer

↓

Database Layer

API Routes Layer

Handles HTTP requests and responses.

Service Layer

Contains business logic and workflows.

Repository Layer

Manages database operations.

Database Layer

Stores and retrieves data.

---

2. Backend Module Design

The FastAPI backend is divided into independent modules.

Authentication Module

Responsibilities:

- User registration
- User login
- JWT generation
- Token validation
- Password hashing
- Password reset requests
- Password reset token verification

Resume Module

Responsibilities:

- Resume upload
- Resume storage
- Resume parsing
- Skill extraction

Interview Module

Responsibilities:

- Interview creation
- Resume-aware interview generation
- Question generation
- Interview session management

Evaluation Module

Responsibilities:

- Answer evaluation
- Score generation
- Feedback generation
- Report creation

Dashboard Module

Responsibilities:

- Performance tracking
- Analytics generation
- History retrieval

Skill Gap Analysis Module

Responsibilities:

- Skill comparison
- Missing skill identification
- Gap report generation

Recommendation Module

Responsibilities:

- Learning path generation
- Resource recommendations
- Improvement suggestions

Coding Assessment Module

Responsibilities:

- Coding question retrieval
- Code submission handling
- Assessment management
- Result generation
- Status tracking

Code Execution Module

Responsibilities:

- Execute submitted code
- Manage execution jobs
- Run test cases
- Capture execution output
- Measure execution time
- Update assessment status
- Store execution results

---

3. Proposed Backend Folder Structure

backend/

│

├── app/

│

├── api/

```
│   ├── auth.py
│   ├── resume.py
│   ├── interview.py
│   ├── evaluation.py
│   ├── dashboard.py
│   ├── skill_gap.py
│   ├── recommendation.py
│   └── coding.py
│
├── services/
│   ├── auth_service.py
│   ├── resume_service.py
│   ├── interview_service.py
│   ├── evaluation_service.py
│   ├── dashboard_service.py
│   ├── skill_gap_service.py
│   ├── recommendation_service.py
│   ├── coding_service.py
│   └── code_execution_service.py
│
├── workers/
│   └── execution_worker.py
│
├── repositories/
│   ├── user_repository.py
│   ├── resume_repository.py
│   ├── interview_repository.py
│   ├── report_repository.py
│   └── coding_repository.py
│
```

```
├── models/
│   ├── user.py
│   ├── password_reset.py
│   ├── resume.py
│   ├── interview.py
│   ├── question.py
│   ├── answer.py
│   ├── report.py
│   └── coding_assessment.py
│
├── schemas/
│
├── core/
│
└── main.py

requirements.txt
```

---

4. Frontend Module Design

The frontend is organized into reusable pages and components.

Pages

- Landing Page
- Login Page
- Register Page
- Forgot Password Page
- Reset Password Page
- Dashboard Page
- Resume Upload Page
- Interview Page
- Report Page
- Skill Gap Page

- Recommendation Page
- Coding Assessment Page

Components

Reusable UI components include:

- Navbar
- Sidebar
- Interview Card
- Analytics Chart
- Resume Upload Widget
- Question Panel
- Feedback Card
- Recommendation Card
- Coding Editor
- Submission Status Panel

---

5. Component Interaction Flow

Interview Generation

Interview Page

↓

Interview API

↓

Interview Service

↓

Resume Retrieval

↓

Gemini Service

↓

Question Generation

↓

Database Storage

↓

Response Returned

Coding Assessment Execution Flow

Code Submission

↓

Coding API

↓

Coding Service

↓

Execution Queue

↓

Execution Worker

↓

Docker Sandbox

↓

Test Case Evaluation

↓

Store Results

↓

Return Status

---

6. AI Integration Design

The AI layer is isolated through dedicated services.

Gemini Service Responsibilities

- Generate interview questions
- Evaluate answers
- Generate feedback
- Analyze skills
- Create recommendations

Benefits

- Easy maintenance
- Vendor replacement flexibility
- Centralized AI logic

---

7. Sandboxed Code Execution Design

To safely execute user-submitted code, InterviewIQ uses an isolated execution architecture.

Execution Workflow

Coding API

↓

Execution Queue

↓

Execution Worker

↓

Docker Sandbox

↓

Result Storage

Security Measures

- Network-disabled containers
- CPU limits
- Memory limits
- Execution timeout
- Read-only file system
- Non-root execution
- Process limits
- Output size restrictions

This architecture protects the platform from malicious code, resource exhaustion attacks, and unauthorized access.

---

8. Error Handling Strategy

The system uses centralized error handling.

Authentication Errors

- Invalid credentials
- Expired tokens
- Invalid reset tokens

Resume Errors

- Unsupported file type
- Parsing failure

AI Errors

- API timeout
- Invalid response

Database Errors

- Connection failure
- Data validation issues

Code Execution Errors

- Compilation error
- Runtime error
- Timeout exceeded
- Memory limit exceeded

---

9. Logging Strategy

The system maintains logs for:

- Authentication events
- Password reset requests
- API requests
- AI operations
- Database activities
- Code execution events
- System errors

Benefits

- Easier debugging
- Monitoring
- Security auditing

---

10. Design Principles Followed

Separation of Concerns

Each module handles a specific responsibility.

Reusability

Components and services are reusable.

Scalability

New features can be added without major restructuring.

Maintainability

Code remains organized and easy to understand.

Modularity

Independent modules reduce complexity.

Outcome of Module 8

A complete low-level design has been created for InterviewIQ, defining backend modules, frontend structure, service responsibilities, repository patterns, AI integration strategy, secure code execution architecture, error handling, and project organization. This design provides a scalable, maintainable, and implementation-ready blueprint for the development of the platform.

Module 9

1. Deployment Architecture

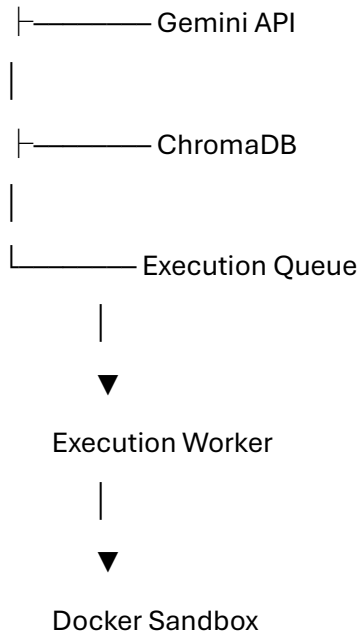
The InterviewIQ system follows a multi-tier deployment architecture.

Users

Next.js Frontend

FastAPI Backend

PostgreSQL



The frontend communicates with the backend through REST APIs, while the backend interacts with databases, AI services, and the secure coding execution infrastructure.

Planned Containers

- Frontend Container
- Backend Container
- ChromaDB Container
- Execution Worker Container
- Docker Sandbox Containers (Created Dynamically)

The coding assessment system uses temporary sandbox containers that are created for each code execution request and automatically destroyed after execution completes.

Secure Code Execution Infrastructure

InterviewIQ supports coding assessments through an isolated execution environment.

Execution Workflow

Code Submission

↓

FastAPI Backend

↓

Execution Queue

↓

Execution Worker

↓

Docker Sandbox

↓

Test Case Execution

↓

Result Storage

Security Measures

- Network-disabled containers
- CPU usage limits
- Memory limits
- Execution timeout limits
- Read-only filesystem
- Non-root execution
- Process restrictions
- Output size limits

This architecture ensures that user-submitted code can be executed safely without affecting the main application infrastructure.

---

11. Security Considerations

Add:

Password Recovery Security

- Password reset tokens are securely generated.
- Only hashed reset tokens are stored.
- Tokens have expiration times.
- Used tokens cannot be reused.

Coding Assessment Security

- User code executes inside isolated Docker containers.
- Containers are automatically destroyed after execution.
- Network access is disabled.
- Resource limits prevent abuse.

---

13. Production Deployment Workflow

Replace with:

Developer

↓

GitHub Repository

↓

GitHub Actions

↓

Frontend Deployment (Vercel)

↓

Backend Deployment (Render)

↓

Database (Supabase / Neon)

↓

ChromaDB Deployment

↓

Execution Worker Deployment

↓

Application Live

---

14. Infrastructure Technology Stack

Replace the table with:

Layer	Technology
Frontend	Next.js
Frontend Hosting	Vercel
Backend	FastAPI
Backend Hosting	Render
Database	PostgreSQL
Database Hosting	Supabase / Neon
Vector Database	ChromaDB
AI Service	Gemini API
Code Execution	Docker Sandbox
Job Processing	Execution Worker
Containerization	Docker
CI/CD	GitHub Actions
Authentication	JWT
Password Security	bcrypt
Monitoring	Grafana / Prometheus (Future)

Updated Outcome of Module 9

Replace the final paragraph with:

Outcome of Module 9

A complete deployment and infrastructure design has been created for InterviewIQ. The architecture defines hosting platforms, deployment workflow, CI/CD strategy, secure coding execution infrastructure, monitoring approach, security measures, and production environment planning. The design supports AI-powered interview generation, coding assessments, secure authentication workflows, and scalable cloud deployment while ensuring reliability, maintainability, and security.

These additions make Module 9 fully aligned with the final InterviewIQ architecture, especially the coding-assessment subsystem and secure Docker-based execution environment.